

Tecniche avanzate

Dizionari avanzati

Tecniche avanzate per lavorare con i dizionari in Python.

Perché ti servono i dizionari avanzati

I dizionari di base li conosci già. Queste tecniche avanzate ti permettono di **crearli in modo più compatto**, gestire chiavi mancanti senza errori, contare occorrenze, e molto altro — cose che tornano utili continuamente nella programmazione reale.

Dictionary comprehension: creare dizionari in una riga

Esattamente come le list comprehension, ma il risultato è un dizionario:

```
# Crea un dizionario {numero: suo_quadrato} per i numeri da 1 a 5
quadrati = {x: x**2 for x in range(1, 6)}
print(quadrati)    # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Crea un dizionario da due liste
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]
registro = {nome: voto for nome, voto in zip(nomi, voti)}
print(registro)    # {'Alice': 8, 'Bob': 7, 'Carlo': 9}

# Con un filtro: solo i pari e i loro quadrati
pari_quadrati = {x: x**2 for x in range(10) if x % 2 == 0}
print(pari_quadrati)    # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

get() : accedere senza rischio di errore

Se accedi a una chiave che non esiste con `dizionario["chiave"]`, Python lancia un `KeyError`. Il metodo `get()` restituisce invece un valore di default.

```
studenti = {"Alice": 8, "Bob": 7}

print(studenti.get("Alice"))      # 8      ← chiave trovata
print(studenti.get("Carlo"))     # None    ← chiave non trovata, nessun
errore
print(studenti.get("Carlo", "N/A")) # N/A    ← chiave non trovata, usa il
valore di default
```

setdefault() : aggiungi una chiave solo se non esiste

Restituisce il valore se la chiave c'è già, altrimenti la aggiunge con il valore di default e restituisce quello.

```
dizionario = {"a": 1}

valore = dizionario.setdefault("a", 99)  # "a" esiste già → non cambia
nulla
print(valore)      # 1
print(dizionario)  # {'a': 1}

valore = dizionario.setdefault("b", 99)  # "b" non esiste → la aggiunge
print(valore)      # 99
print(dizionario)  # {'a': 1, 'b': 99}
```

Un uso pratico: costruire un dizionario dove i valori sono liste, senza dover controllare se la chiave esiste già:

```
# Raggruppa le parole per la loro prima lettera
parole = ["mela", "banana", "more", "mandarino"]
raggruppate = {}

for parola in parole:
    lettera = parola[0]
    raggruppate.setdefault(lettera, []).append(parola) # Se la chiave non
    c'è, crea una lista vuota

print(raggruppate)
# {'m': ['mela', 'more', 'mandarino'], 'b': ['banana']}
```

Counter : conta le occorrenze automaticamente

Dal modulo `collections`, `Counter` è un dizionario speciale pensato per contare quante volte appare ogni elemento.

```
from collections import Counter

# Conta quante volte appare ogni frutto
frutta = ["mela", "banana", "mela", "uva", "banana", "mela"]
contatore = Counter(frutta)
print(contatore)
# Counter({'mela': 3, 'banana': 2, 'uva': 1})

print(contatore["mela"])      # 3 - occorrenze di "mela"
print(contatore["kiwi"])      # 0 - chiave non trovata → 0 (non
errore!)

print(contatore.most_common(2)) # [('mela', 3), ('banana', 2)] - i 2 più
frequenti

# Conta le lettere in una stringa
lettere = Counter("mississippi")
print(lettere)
# Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
```

defaultdict : valori di default automatici

Un altro strumento di `collections`. Con `defaultdict` non devi mai preoccuparti di controllare se una chiave esiste: se non c'è, viene creata automaticamente con il valore di default che hai scelto.

```
from collections import defaultdict

# defaultdict(list) crea automaticamente una lista vuota per le chiavi
nuove
gruppi = defaultdict(list)

studenti = [("Alice", "3A"), ("Bob", "3B"), ("Carlo", "3A")]
for nome, classe in studenti:
    gruppi[classe].append(nome)  # Non serve controllare se "3A" esiste
    già!

print(dict(gruppi))
# {'3A': ['Alice', 'Carlo'], '3B': ['Bob']}
```

Unire due dizionari

```
d1 = {"a": 1, "b": 2}
d2 = {"b": 3, "c": 4}

# Python 3.9+: operatore | (come l'unione degli insiemi)
unito = d1 | d2
print(unito)  # {'a': 1, 'b': 3, 'c': 4} ← "b" di d2 sovrascrive "b" di d1

# Alternativa (funziona in tutte le versioni)
unito2 = {**d1, **d2}
print(unito2)  # {'a': 1, 'b': 3, 'c': 4}
```

Invertire un dizionario (chiavi ↔ valori)

```
originale = {"a": 1, "b": 2, "c": 3}
invertito = {v: k for k, v in originale.items()} # Scambia chiavi e valori
print(invertito) # {1: 'a', 2: 'b', 3: 'c'}
```

Attenzione: funziona solo se i valori sono unici. Se due chiavi avevano lo stesso valore, una verrà sovrascritta.

Ordinare un dizionario

I dizionari in Python 3.7+ mantengono l'ordine di inserimento. Ma se vuoi ordinarli per chiave o per valore:

```
voti = {"Alice": 8, "Carlo": 9, "Bob": 7}

# Ordina per chiave (alfabetico)
per_nome = dict(sorted(voti.items()))
print(per_nome) # {'Alice': 8, 'Bob': 7, 'Carlo': 9}

# Ordina per valore, dal più alto al più basso
per_voto = dict(sorted(voti.items(), key=lambda x: x[1], reverse=True))
print(per_voto) # {'Carlo': 9, 'Alice': 8, 'Bob': 7}
```

Liste avanzate

List comprehension e tecniche avanzate per le liste in Python.

Perché è utile usare le liste avanzate

Sai già usare le liste di base. Ora impari le tecniche che rendono il codice Python **più corto, più leggibile e più efficiente** — strumenti che i programmatori Python usano ogni giorno.

List comprehension: creare liste in una riga

La **list comprehension** è un modo compatto per costruire una lista a partire da un'altra sequenza, tutto in una sola riga.

Il problema che risolve

```
# Metodo tradizionale: 4 righe per creare una lista di quadrati
quadrati = []
for x in range(1, 6):
    quadrati.append(x ** 2)
print(quadrati)    # [1, 4, 9, 16, 25]

# Con list comprehension: 1 sola riga, stesso risultato
quadrati = [x ** 2 for x in range(1, 6)]
print(quadrati)    # [1, 4, 9, 16, 25]
```

Leggi la list comprehension come: "x al quadrato, per ogni x in range(1, 6)".

Con un filtro (**if**)

Puoi aggiungere una condizione per includere solo certi elementi:

```
# Solo i numeri pari tra 0 e 9
pari = [x for x in range(10) if x % 2 == 0]
print(pari)    # [0, 2, 4, 6, 8]

# Solo le parole di più di 4 lettere
parole = ["mela", "banana", "uva", "ciliegia", "kiwi"]
lunghe = [p for p in parole if len(p) > 4]
print(lunghe)    # ['banana', 'ciliegia']
```

Con `if-else` (trasformazione condizionale)

Puoi trasformare ogni elemento in modo diverso a seconda di una condizione:

```
# Classifica ogni numero come "pari" o "dispari"
numeri = range(6)
classificati = ["pari" if x % 2 == 0 else "dispari" for x in numeri]
print(classificati)  # ['pari', 'dispari', 'pari', 'dispari', 'pari', 'dispari']
```

List comprehension annidate (per le matrici)

Puoi usare una list comprehension dentro un'altra per lavorare con strutture a griglia (righe e colonne):

```
# Creare una matrice 3x3 (lista di liste)
matrice = [[j for j in range(3)] for i in range(3)]
print(matrice)  # [[0, 1, 2], [0, 1, 2], [0, 1, 2]]

# "Appiattare" una matrice in una lista semplice
matrice = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
piatta = [elem for riga in matrice for elem in riga]
print(piatta)  # [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Ordinare con `sorted()` e una chiave personalizzata

Per default `sorted()` ordina in ordine alfabetico o numerico. Con il parametro `key` puoi scegliere tu **in base a cosa** ordinare.

```
studenti = [("Alice", 16), ("Carlo", 15), ("Bob", 17)]

# Ordina per età (il secondo elemento della coppia)
per_eta = sorted(studenti, key=lambda s: s[1])
print(per_eta)  # [('Carlo', 15), ('Alice', 16), ('Bob', 17)]

# Ordina per nome (il primo elemento della coppia)
per_nome = sorted(studenti, key=lambda s: s[0])
print(per_nome)  # [('Alice', 16), ('Bob', 17), ('Carlo', 15)]

# Ordine dal più grande al più piccolo
decescente = sorted(studenti, key=lambda s: s[1], reverse=True)
print(decescente)  # [('Bob', 17), ('Alice', 16), ('Carlo', 15)]
```

map() e filter() : applicare funzioni a liste

```
numeri = [1, 2, 3, 4, 5]

# map(): applica una funzione a ogni elemento e restituisce i risultati
doppi = list(map(lambda x: x * 2, numeri))
print(doppi)  # [2, 4, 6, 8, 10]

# filter(): tieni solo gli elementi che soddisfano la condizione
grandi = list(filter(lambda x: x > 3, numeri))
print(grandi)  # [4, 5]
```

:::note Spesso le list comprehension sono più leggibili di `map` e `filter`. Usa quella forma che trovi più chiara. :::

zip() : scorrere più liste insieme

`zip()` combina due o più liste in coppie (o triplete, ecc.), permettendoti di scorrerle tutte insieme.

```
nomi = ["Alice", "Bob", "Carlo"]
voti = [8, 7, 9]
citta = ["Roma", "Milano", "Napoli"]

# Scorrere due liste in parallelo
for nome, voto in zip(nomi, voti):
    print(f"{nome}: {voto}")
# → Alice: 8
# → Bob: 7
# → Carlo: 9

# Creare una lista di coppie
coppie = list(zip(nomi, voti, citta))
print(coppie)
# → [('Alice', 8, 'Roma'), ('Bob', 7, 'Milano'), ('Carlo', 9, 'Napoli')]
```

Operazioni comuni sulle liste

```
numeri = [3, 1, 4, 1, 5, 9, 2, 6]

print(sum(numeri))      # 31 – somma di tutti i valori
print(max(numeri))      # 9 – il valore più grande
print(min(numeri))      # 1 – il valore più piccolo
print(len(numeri))      # 8 – quanti elementi ci sono

# Calcolare la media
media = sum(numeri) / len(numeri)
print(f"Media: {media:.2f}")  # Media: 3.88

# Rimuovere i duplicati mantenendo l'ordine originale
unici = list(dict.fromkeys(numeri))
print(unici)  # [3, 1, 4, 5, 9, 2, 6]
```

Stringhe avanzate

Metodi avanzati per la manipolazione delle stringhe in Python.

Qual è la funzione delle stringhe avanzate

Le stringhe sono ovunque: nomi, messaggi, dati da file, risposte dal web. Python ha decine di strumenti pronti per cercare, modificare, tagliare e unire testo. In questa pagina vediamo quelli più utili nella pratica.

Cercare testo dentro una stringa

`find()` — trova la posizione di una sottostringa

```
testo = "Ciao, mi chiamo Alice e vivo a Roma."

print(testo.find("Alice"))    # 15 → "Alice" inizia alla posizione 15
print(testo.find("Mario"))    # -1 → non trovato (restituisce -1 invece di
                               # errore)
print(testo.rfind("a"))       # 31 → posizione dell'ULTIMA "a"
```

`count()` — conta quante volte appare qualcosa

```
testo = "banana"
print(testo.count("a"))      # 3
print(testo.count("na"))    # 2
```

`replace()` — sostituisci una parte del testo

```
testo = "Ciao mondo! Ciao Python!"
print(testo.replace("Ciao", "Salve"))      # Salve mondo! Salve Python!
print(testo.replace("Ciao", "Salve", 1))   # Salve mondo! Ciao Python!
(solo la prima)
```

Tagliare e unire

`split()` — divide una stringa in una lista di pezzi

```
frase = "mela banana ciliegia uva"
parole = frase.split()                  # Divide sugli spazi (di default)
print(parole)    # ['mela', 'banana', 'ciliegia', 'uva']

# Dividere su un altro separatore
dati = "Alice,16,Roma"
campi = dati.split(",")                 # Divide sulle virgole
print(campi)    # ['Alice', '16', 'Roma']

# Limitare il numero di divisioni
print("a:b:c:d".split(":", 2))          # ['a', 'b', 'c:d'] ← si ferma dopo 2
tagli
```

`splitlines()` — divide su ogni fine riga

```
testo = "Prima riga\nSeconda riga\nTerza riga"
righe = testo.splitlines()
print(righe)    # ['Prima riga', 'Seconda riga', 'Terza riga']
```

`join()` — unisce una lista di stringhe in una sola

`join()` è l'opposto di `split()` : prende una lista e la incolla insieme usando un separatore.

```
parole = ["mela", "banana", "ciliegia"]
print(", ".join(parole))    # mela, banana, ciliegia
print(" - ".join(parole))   # mela - banana - ciliegia
print("".join(parole))      # melebananaciliegia
```

Pulizia del testo

Rimuovere spazi in eccesso

```
testo = "    ciao mondo    "
print(testo.strip())      # "ciao mondo"      ← rimuove da entrambi i lati
print(testo.lstrip())     # "ciao mondo    "  ← rimuove solo a sinistra
print(testo.rstrip())     # "    ciao mondo"  ← rimuove solo a destra

# Rimuovere caratteri specifici (non solo spazi)
testo2 = "###ciao###"
print(testo2.strip("#"))  # "ciao"
```

Cambiare maiuscole e minuscole

```
testo = "ciao mondo"
print(testo.upper())      # CIAO MONDO
print(testo.lower())      # ciao mondo
print(testo.capitalize()) # Ciao mondo ← solo la prima lettera
                           maiuscola
print(testo.title())      # Ciao Mondo ← prima lettera di ogni parola
                           maiuscola
print(testo.swapcase())   # CIAO MONDO ← inverte: minuscole↔maiuscole
```

Verificare il contenuto di una stringa

Questi metodi restituiscono `True` o `False` :

```
print("hello".islower())    # True ← tutto minuscolo
print("HELLO".isupper())    # True ← tutto maiuscolo
print("Hello World".istitle()) # True ← ogni parola inizia con maiuscola

print("abc123".isalnum())   # True ← solo lettere e numeri
print("abc".isalpha())      # True ← solo lettere
print("123".isdigit())      # True ← solo cifre
print(" ".isspace())        # True ← solo spazi
```

Allineamento e riempimento

Utili per creare tabelle o output ben formattato:

```
print("42".zfill(5))          # "00042"    ← riempie con zeri a
sinistra
print("ciao".ljust(10))       # "ciao      " ← allinea a sinistra (spazi
a destra)
print("ciao".rjust(10))       # "      ciao" ← allinea a destra (spazi a
sinistra)
print("ciao".center(10))      # "   ciao   " ← centra
print("ciao".center(10, "-")) # "---ciao---" ← centra con carattere
personalizzato
```

Dividere in tre parti con `partition()`

`partition()` divide la stringa in esattamente tre parti: prima del separatore, il separatore stesso, dopo il separatore.

```
testo = "nome: Alice"
prima, sep, dopo = testo.partition(":")
print(prima)    # "nome"
print(sep)      # ":"
print(dopo)     # " Alice"
```

Esempio pratico: leggere dati separati da virgole

Un formato molto comune per i dati è il CSV (valori separati da virgole). Ecco come estrarre le informazioni da una riga:

```
riga_csv = "Alice,16,Roma,8.5"
campi = riga_csv.split(",") # Divide sulle virgole

nome = campi[0].strip()      # Rimuove eventuali spazi
eta = int(campi[1].strip())  # Converte in numero intero
citta = campi[2].strip()
voto = float(campi[3].strip()) # Converte in numero decimale

print(f"{nome} ha {eta} anni, vive a {citta} e ha voto {voto}")
# → Alice ha 16 anni, vive a Roma e ha voto 8.5
```

Array (Liste come array)

Come usare le liste Python come array e quando usare librerie specifiche.

Perché è utile usare gli array in Python

In altri linguaggi come C++ o Java, un **array** è una sequenza di elementi tutti dello stesso tipo, con una posizione numerata che parte da 0. In Python, le **liste** svolgono questo ruolo nella maggior parte dei casi — e sono ancora più flessibili.

Le liste come array

Per uso generico, le liste Python si comportano esattamente come array:

```
# Una sequenza di numeri interi, con indici 0, 1, 2, 3, 4
numeri = [10, 20, 30, 40, 50]

# Accedere a un elemento tramite il suo indice (la posizione)
print(numeri[0])    # 10 ← primo elemento
print(numeri[2])    # 30 ← terzo elemento
print(numeri[-1])   # 50 ← ultimo elemento (si conta da destra)

# Modificare un elemento
numeri[1] = 25
print(numeri)       # [10, 25, 30, 40, 50]
```

Il modulo `array` : più efficiente per grandi quantità di numeri

Python ha un modulo chiamato `array` che offre array più efficienti in memoria rispetto alle liste normali. A differenza delle liste, però, possono contenere solo elementi dello **stesso tipo**.

```
# Crea un array di interi ('i' = integer)
numeri = array.array('i', [1, 2, 3, 4, 5])

print(numeri)          # array('i', [1, 2, 3, 4, 5])
print(numeri[0])       # 1

numeri.append(6)        # Aggiunge un elemento in fondo
print(numeri)          # array('i', [1, 2, 3, 4, 5, 6])

numeri.insert(2, 99)    # Inserisce 99 alla posizione 2
print(numeri)          # array('i', [1, 2, 99, 3, 4, 5, 6])

numeri.remove(99)       # Rimuove la prima occorrenza di 99
print(numeri)          # array('i', [1, 2, 3, 4, 5, 6])
```

I codici tipo più usati:

Codice	Tipo	Descrizione
'i'	int	Numero intero
'l'	long	Numero intero lungo
'f'	float	Numero decimale (singola precisione)
'd'	double	Numero decimale (doppia precisione)

Matrici: array a due dimensioni

Una **matrice** è come una tabella con righe e colonne. In Python si rappresenta come una lista di liste:

```
# Una tabella 3x3
matrice = [
    [1, 2, 3], # riga 0
    [4, 5, 6], # riga 1
    [7, 8, 9]  # riga 2
]

# Accedere a un elemento: prima la riga, poi la colonna
print(matrice[0][0]) # 1 ← riga 0, colonna 0
print(matrice[1][2]) # 6 ← riga 1, colonna 2
print(matrice[2])    # [7, 8, 9] ← tutta la riga 2

# Modificare un elemento
matrice[1][1] = 99
print(matrice[1])    # [4, 99, 6]
```

Scorrere un array (o una matrice) con **for**

```
numeri = [10, 20, 30, 40, 50]

# Scorrere tutti gli elementi
for n in numeri:
    print(n)

# Scorrere con l'indice
for i in range(len(numeri)):
    print(f"numeri[{i}] = {numeri[i]}")

# Scorrere una matrice riga per riga, elemento per elemento
matrice = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
for riga in matrice:
    for elemento in riga:
        print(elemento, end=" ")    # Stampa sulla stessa riga
    print()    # Va a capo dopo ogni riga
```

Operazioni comuni

```
numeri = [5, 2, 8, 1, 9, 3]

print(max(numeri))    # 9 - il valore più grande
print(min(numeri))    # 1 - il valore più piccolo
print(sum(numeri))    # 28 - la somma di tutti
print(len(numeri))    # 6 - quanti elementi ci sono
print(sorted(numeri)) # [1, 2, 3, 5, 8, 9] - copia ordinata
```

Quando usare NumPy invece delle liste?

Le liste Python vanno bene per la maggior parte degli usi. Ma se devi fare **calcoli matematici su grandi quantità di numeri** — come la media di 10.000 valori, moltiplicare matrici, o applicare funzioni statistiche — usa la libreria **NumPy**.

NumPy fa le stesse cose delle liste, ma è molto più veloce per i calcoli numerici intensivi. Lo vedremo nella sezione dedicata all'ecosistema Python.